BulkPay ERP – Plateforme de paiements en masse

Hackathon Bitnob · Équipe de 5 personnes · Version 1.0 · 17 octobre 2026

Thème : faciliter les paiements en ligne de masse pour les entreprises africaines.

1. Contexte général

En Afrique, les entreprises qui travaillent à l'échelle du continent (PME, startups, marketplaces, plateformes de livraison, agences, ONG) doivent régulièrement payer de nombreuses personnes réparties dans plusieurs pays : salariés, freelances, vendeurs, livreurs, prestataires, bénéficiaires d'aides ou de bourses.

Ces paiements passent aujourd'hui par un patchwork de moyens : virements bancaires locaux ou internationaux, mobile money envoyé un numéro à la fois, plateformes différentes selon le pays. Chaque pays a ses devises, ses opérateurs et ses règles.

En parallèle, les stablecoins et le Bitcoin (notamment via le réseau Lightning) offrent une infrastructure de transfert rapide, programmable et sans frontières. Bitnob propose des API qui permettent de relier cette infrastructure aux rails locaux : paiements vers comptes bancaires et mobile money, wallets, factures. C'est la brique technique sur laquelle repose notre projet.

2. Problématique

Payer des dizaines ou des centaines de personnes dans plusieurs pays reste lent, coûteux et difficile à suivre.

- **Frais élevés** : commissions bancaires, frais de change et frais de transfert cumulés à chaque paiement.
- **Processus manuels** : saisie des bénéficiaires un par un, copier-coller de numéros mobile money, risques d'erreurs de montant ou de destinataire.
- **Multiplicité des outils** : une solution par pays ou par moyen de paiement, sans vue d'ensemble.
- **Comptabilité compliquée** : pas de rapprochement simple entre ce qui a été payé, à qui, pourquoi et à quel taux de change.
- **Manque de visibilité** : il est difficile de répondre à des questions simples comme « combien avons-nous payé ce mois-ci ? » ou « quelle part de nos dépenses a été réglée en Bitcoin ? ».
- **Dépenses en ligne mal maîtrisées** : la publicité et les logiciels (Meta Ads, Notion, Figma, Canva) se règlent souvent avec une carte unique, sans budget cloisonné par usage.

Question centrale du projet : comment permettre à une organisation de payer en une seule opération un grand nombre de bénéficiaires dans plusieurs pays, tout en gardant une comptabilité claire et des réponses immédiates à ses questions financières ?

3. Objectifs

Objectif général : offrir à une organisation un outil unique, accessible en ligne, pour payer, encaisser, budgétiser et analyser ses flux financiers en s'appuyant sur les API Bitnob.

Objectifs spécifiques :

- Réduire le temps de traitement d'une paie ou d'un lot de commissions de plusieurs heures à quelques minutes.
- Éviter les erreurs grâce à la validation des données avant l'envoi.
- Donner une vue claire et en temps réel de l'argent entrant et sortant.
- Cloisonner les dépenses par usage grâce à des enveloppes budgétaires, chacune adossée à une carte virtuelle dédiée.
- Rendre l'analyse financière accessible à tous grâce à un assistant en langage naturel.

4. Solution proposée

BulkPay ERP est une plateforme web de gestion financière légère, un mini-ERP centré sur les paiements de masse. Un administrateur peut y importer une liste de bénéficiaires, vérifier un récapitulatif, lancer les paiements en un clic, suivre leur statut, émettre des factures en Bitcoin et interroger ses données en langage naturel.

Les cinq modules

Module 1 – Paiements de masse par CSV

- L'administrateur importe un fichier CSV (nom, numéro ou compte, montant, devise, pays).
- L'application valide chaque ligne : montants, devises, pays supportés, doublons, format des numéros.
- Un récapitulatif s'affiche avant l'envoi : nombre de bénéficiaires, montant total, frais estimés, pays couverts.
- Le lot est lancé en un clic ; chaque paiement passe par les étapes Bitnob (devis, initialisation, finalisation).
- Le suivi est en temps réel : en cours, réussi, échoué, avec possibilité de relancer les échecs et d'exporter un rapport.

Module 2 – Tableau de bord

- Indicateurs clés : volume total payé, nombre de paiements, bénéficiaires uniques, taux de réussite, frais cumulés.
- Graphiques : évolution dans le temps, répartition par pays, par devise et par enveloppe budgétaire.
- Alertes sur les paiements échoués ou les budgets proches de leur limite.

Module 3 – Réception de Bitcoin par QR code

- L'administrateur crée une facture (montant, description, client) ; la plateforme génère une facture Bitcoin sur le réseau Lightning.
- La facture s'affiche sous forme de QR code que le client scanne avec son portefeuille pour payer en Bitcoin.

- Le statut se met à jour automatiquement à la réception du paiement : en attente, payée, expirée.

Module 4 – Enveloppes budgétaires par carte virtuelle

- L'administrateur crée des enveloppes, par exemple « Meta Ads – 100 USD par mois », « Outils (Notion, Figma, Canva) – 50 USD par mois » et « Formation et livres – 30 USD par mois ».
- Chaque enveloppe crée une carte virtuelle Bitnob dédiée, financée à hauteur du budget depuis le solde de l'entreprise.
- L'utilisateur ne voit jamais les numéros de carte : seulement le nom du budget, le solde restant et les dépenses.
- La carte est gelée automatiquement quand le budget est épuisé ; une alerte prévient quand il reste peu (par exemple « Il te reste 10 USD sur ton budget Meta Ads »).
- Chaque début de mois, les enveloppes sont rechargées de leur budget.

Module 5 – Assistant IA analytique

- Un chatbot interroge la base de données pour répondre à des questions comme : « Combien avons-nous payé ce mois-ci ? », « Quelles dépenses ont été réglées en Bitcoin ? », « Quel pays reçoit le plus de paiements ? ».
- Chaque réponse affiche aussi la requête utilisée, pour la transparence.
- L'accès est strictement en lecture seule et limité à des vues prédéfinies.

5. Utilisateurs cibles et scénario de démonstration

Utilisateurs cibles : PME et startups panafricaines, marketplaces et plateformes de livraison, agences et équipes de freelances, ONG et programmes de bourses.

Scénario de démonstration : une marketplace de livraison doit payer ses commissions du mois à 50 livreurs au Nigeria, au Ghana et au Sénégal.

1. L'administrateur crée trois enveloppes (Meta Ads, Outils, Formation) ; chacune obtient sa carte virtuelle financée.
2. Il importe le CSV de 50 lignes ; l'application détecte et signale deux numéros invalides.
3. Il consulte le récapitulatif (total, frais, pays), puis lance le lot en un clic.
4. Les statuts se mettent à jour en direct ; un échec est relancé.
5. Le tableau de bord se met à jour.
6. Il demande à l'assistant : « Combien avons-nous payé au Ghana ce mois-ci ? » et obtient la réponse immédiatement.
7. Une dépense sur l'enveloppe Meta Ads déclenche une alerte de solde bas, puis le gel de la carte à l'épuisement du budget.
8. Il émet une facture, affiche le QR code Bitcoin et la voit passer à « payée » après le paiement.

6. Architecture technique

Vue d'ensemble : le frontend appelle le backend ; le backend dialogue avec Bitnob et lit ou écrit dans la base PostgreSQL ; le module IA interroge la base en lecture seule ; Bitnob renvoie des événements au backend par webhooks.

Composants :

- **Frontend web** (React ou Next.js) : pages d'import, de suivi, de factures, d'enveloppes budgétaires, de tableau de bord et de chat.
- **Backend API** (FastAPI, Docker) : validation, orchestration des paiements et des cartes, intégration Bitnob avec authentification signée, réception des webhooks.
- **Base de données** (PostgreSQL) : utilisateurs, lots, bénéficiaires, paiements, budgets, transactions de cartes, alertes, factures, plus des vues SQL pour l'analyse.
- **Module IA** : agent qui traduit une question en requête SQL sécurisée et formule la réponse.
- **Infrastructure** : Docker Compose pour lancer l'ensemble ; GitHub pour le code et l'intégration continue.

Principes de conception :

- Le secret Bitnob reste uniquement côté backend.
- Le succès d'un paiement n'est confirmé que par l'événement de livraison reçu par webhook.
- Les paiements sont idempotents : relancer un lot ne doit jamais payer deux fois la même personne.
- Les montants sont stockés avec une précision décimale suffisante pour les crypto-actifs.
- Les numéros de carte ne sont ni stockés ni affichés : seuls l'identifiant de carte et les quatre derniers chiffres sont conservés.

7. Périmètre et risques

Dans le périmètre du MVP : import CSV et validation, exécution du lot, suivi par webhooks, enveloppes budgétaires par carte virtuelle avec gel automatique et alertes, factures Bitcoin par QR code, tableau de bord, assistant IA en lecture seule, authentification administrateur simple.

Hors périmètre : gestion multi-entreprises, rôles avancés, conformité réglementaire complète, application mobile, comptabilité légale détaillée.

Hypothèses et risques à surveiller :

- La création d'une carte exige la vérification d'identité du titulaire (Card KYC) ; son fonctionnement en sandbox est le premier test à faire, car tout le module d'enveloppes en dépend.
- Les cartes sont prépayées, en dollars américains et utilisables en ligne uniquement : la démonstration reste en USD.
- En sandbox, les dépenses sur les cartes devront être simulées ou fournies par des transactions de test (à confirmer) ; les pays et rails disponibles peuvent aussi limiter la démonstration.

- Le rechargement mensuel et les alertes relèvent de notre code : tâche planifiée et seuils vérifiés à chaque événement de carte.
- Les factures Bitcoin reposent sur le réseau Lightning ; les endpoints exacts et l'événement de paiement restent à confirmer.
- Les devis de change expirent vite : l'exécution doit être rapide ou refaite.
- Un assistant qui génère du SQL doit être strictement encadré (lecture seule, vues, validation).

8. Valeur ajoutée

- **Gain de temps** : un lot de dizaines de paiements en quelques minutes.
- **Fiabilité** : validation avant envoi, suivi précis, relance des échecs.
- **Contrôle** : un budget par carte virtuelle, gel automatique à l'épuisement, traçabilité complète.
- **Transparence** : un assistant qui répond à des questions financières sans compétences techniques.
- **Adaptabilité** : le même socle sert pour les salaires, commissions, remboursements, aides ou bourses.

9. Organisation de l'équipe et répartition des tâches

Chaque personne est propriétaire d'un dossier du dépôt et ne modifie que celui-ci ; tout besoin sur le dossier d'un autre passe par une issue ou une pull request.

Personne 1 – Backend API (dossier backend)

- Maintenir le client Bitnob signé et le flux de paiement (devis, initialisation, finalisation).
- Importer et valider le CSV ; créer le lot et les paiements.
- Exécuter les lots en tâche de fond avec idempotence et relance des échecs.
- Recevoir les webhooks Bitnob et mettre à jour les statuts.
- Implémenter les enveloppes budgétaires : vérification d'identité du titulaire, création et alimentation de la carte, gel et dégel, lecture des transactions, rechargement mensuel et alertes de seuil.
- Implémenter les factures Bitcoin : création Lightning, génération du QR code, mise à jour du statut au paiement.
- Exposer le résumé du tableau de bord à partir des vues de la base.
- Publier et tenir à jour le contrat d'API (OpenAPI et document de référence).

Personne 2 – Base de données et tableau de bord (dossier db)

- Concevoir le schéma : utilisateurs, lots, bénéficiaires, paiements, budgets, transactions de cartes, alertes, factures.
- Écrire les migrations et le script de données de démonstration réalistes.
- Créer les vues SQL : indicateurs clés, paiements par mois, par pays, par devise, par enveloppe budgétaire.
- Créer un utilisateur PostgreSQL en lecture seule pour l'assistant IA.
- Fournir au frontend les données nécessaires au tableau de bord (avec la personne 1).
- Publier la documentation du schéma.

Personne 3 – Assistant IA (dossier ai)

- Choisir le modèle de langage et gérer sa clé d'accès.
- Construire l'agent : question en langage naturel, requête SQL, réponse rédigée.
- Mettre en place les garde-fous : une seule instruction de lecture, limite de lignes, délai maximum, accès aux vues uniquement.
- Exposer l'endpoint de chat qui renvoie la réponse, la requête et les données.
- Préparer un jeu de questions de test pour la démonstration.
- Gérer l'historique de conversation et les questions ambiguës.

Personne 4 – Design et frontend (dossier frontend)

- Définir l'identité visuelle, les maquettes et le système de composants.
- Concevoir les parcours : import CSV, récapitulatif, suivi des lots, factures avec QR code, enveloppes budgétaires, tableau de bord, chat.
- Développer les composants et les pages, d'abord sur des données factices.
- Soigner l'ergonomie, les états de chargement et d'erreur, et l'adaptation aux mobiles.

Personne 5 – Dépôt GitHub, intégration et frontend

- Créer le dépôt, la structure en dossiers, les branches protégées et les modèles de pull request.
- Mettre en place l'intégration continue : tests et vérifications de style.
- Configurer Docker Compose pour lancer toute l'application avec une seule commande.
- Relire les pull requests et résoudre les conflits.
- Brancher les pages du frontend aux vrais endpoints avec la personne 4.
- Rédiger le README, préparer le scénario de démonstration, les diapositives et la vidéo.

Tâches partagées

- Kickoff : validation de l'architecture, rédaction du contrat d'API et du schéma de base (toute l'équipe).
- Point d'intégration à mi-parcours : chacun présente ce qui fonctionne.
- Tests de bout en bout du scénario de démonstration (toute l'équipe).
- Répétition de la présentation finale.

10. Planning indicatif

- **Phase 0 – Kickoff (1 à 2 heures)** : valider l'architecture, écrire le contrat d'API et le schéma de base, créer le dépôt et l'environnement Docker.
- **Phase 1 – Travail parallèle (environ 40 % du temps)** : chacun avance dans son dossier avec des données factices ; le frontend utilise des réponses simulées.
- **Phase 2 – Intégration (environ 30 %)** : le frontend passe aux vrais endpoints, l'IA se branche sur les vraies vues, test du flux complet en sandbox Bitnob.
- **Phase 3 – Finition (environ 30 %)** : correction des bugs, amélioration du design, jeu de données de démonstration, README, présentation et vidéo.

11. Règles de collaboration

- Branches protégées : une branche principale pour la démonstration et une branche d'intégration.
- Une branche par fonctionnalité, nommée par exemple feat/backend-bulk ou feat/ai-chat.
- Pull request obligatoire avec au moins une relecture ; commits petits et fréquents.
- Les secrets ne sont jamais commités : seul le fichier d'exemple des variables d'environnement l'est.
- Un canal de discussion unique et un point rapide à heures fixes.
- Toute modification du contrat d'API ou du schéma est annoncée à l'équipe avant d'être fusionnée.

12. Critères de réussite

- Le scénario de démonstration se déroule de bout en bout sans intervention manuelle.
- Un lot de paiements est exécuté en sandbox Bitnob et ses statuts se mettent à jour automatiquement.
- Une enveloppe budgétaire crée sa carte, la reçoit financée, enregistre une dépense et se gèle à l'épuisement du budget.
- Une facture Bitcoin génère un QR code et passe à « payée » à la réception du paiement.
- Le tableau de bord reflète les opérations réalisées.
- L'assistant répond correctement à au moins cinq questions analytiques préparées.
- Le projet se lance avec une seule commande et dispose d'une documentation claire.